UNICID — UNIVERSIDADE CIDADE DE SÃO PAULO

F3 — Transformador de estrutura tipo JSON

Paradigma Funcional · Elixir · glot.io

Paradigmas de Linguagens de Programação | Prof. Cid Rodrigues De Andrade

Seminário de entrega · Grupo 5 alunos

GRUPO

Integrantes

Gabriel Seren Franco	RGM 33696896
Gian Lucca Panzone	RGM 34014781
Giovanne Renato da Silva Vieira	RGM 33483019
Lucas Wu Hai Hua	RGM 33251380
Robson Luís Pereira de Almeida Mesquita	RGM 32871571

Professor: Cid Rodrigues De Andrade
Disciplina: Paradigmas de Linguagens de Programação

TEMA F3

O que o tema pede

Dada uma estrutura de dados **aninhada** (semelhante a JSON — mapas dentro de mapas, listas dentro de mapas), escrever funções **recursivas** que:

- **Renomeiem** chaves em qualquer profundidade
- **Extraiam** valores associados a uma chave
- **Achatem** a árvore (flatten) em um mapa plano

> *Usar pattern matching sobre a estrutura, em vez de ifs explícitos checando o tipo de cada nó.*

PARADIGMA

Por que o paradigma funcional?

Ideias centrais

- Funções como unidade principal
- Imutabilidade dos dados
- Recursão em vez de loops mutáveis
- Composição de transformações

Encaixe no tema F3

- Árvore JSON = estrutura recursiva
- Cada nó vira um caso de pattern matching
- Transformação pura: entrada → saída
- Sem estado global alterado

LINGUAGEM

Por que Elixir?

- Linguagem funcional rodando na VM Erlang (BEAM)
- **Pattern matching** nativo em cláusulas de função
- Mapas (`%{}`) e listas modelam JSON com naturalidade
- Guards (`when is_map`, `when is_list`) selecionam a cláusula certa
- Ambiente livre e rápido no **glot.io** para demonstrar

```elixir
# Três cláusulas = três "tipos" de nó — sem if/else
def renomear_chaves(mapa, regras) when is_map(mapa), do: ...
def renomear_chaves(lista, regras) when is_list(lista), do: ...
def renomear_chaves(valor, _regras), do: valor  # folha
```

CONCEITO-CHAVE

Pattern matching × ifs de tipo

Abordagem imperativa

```
if is_map(no) do
  # tratar mapa
else if is_list(no) do
  # tratar lista
else
  # folha
end
```

vs

Abordagem funcional (Elixir)

```
def f(no) when is_map(no), do: ...
def f(no) when is_list(no), do: ...
def f(no), do: ...
```

O runtime escolhe a cláusula. O código declara o quê, não encadeia se for tipo X.

MODELO

Estrutura de exemplo

```
%{
  "usuario" => %{
    "nome" => "Ana Silva",
    "idade" => 28,
    "contato" => %{
      "email" => "ana@email.com",
      "telefone" => "11999998888"
    }
  },
  "pedidos" => [
    %{"id" => 1, "produto" => "Livro", "preco" => 45.90},
    %{"id" => 2, "produto" => "Caneta", "preco" => 3.50}
  ],
  "ativo" => true
}
```

Mapas aninhados + lista de mapas — o mesmo formato mental de um JSON.

FUNÇÃO 1

Renomear chaves

Percorre a árvore; se a chave estiver no mapa de regras, troca o nome e continua nos filhos.

```
def renomear_chaves(mapa, regras) when is_map(mapa) do
  mapa
  |> Enum.map(fn {chave, valor} ->
    nova_chave = Map.get(regras, chave, chave)
    {nova_chave, renomear_chaves(valor, regras)}
  end)
  |> Map.new()
end

def renomear_chaves(lista, regras) when is_list(lista) do
  Enum.map(lista, fn item -> renomear_chaves(item, regras) end)
end

def renomear_chaves(valor, _regras), do: valor
```

Ex.: `"nome"` → `"name"`, `"usuario"` → `"user"` em toda a estrutura.

FUNÇÃO 2

Extrair valores

Coleta todos os valores de uma chave, em qualquer profundidade — inclusive dentro de listas.

```
def extrair_valores(mapa, chave_alvo) when is_map(mapa) do
  Enum.flat_map(mapa, fn
    {^chave_alvo, valor} ->
      [valor | extrair_valores(valor, chave_alvo)]
    {_outra, valor} ->
      extrair_valores(valor, chave_alvo)
  end)
end

def extrair_valores(lista, chave_alvo) when is_list(lista) do
  Enum.flat_map(lista, fn item -> extrair_valores(item, chave_alvo) end)
end

def extrair_valores(_valor, _chave_alvo), do: []
```

`^chave_alvo` (pin) exige igualdade no pattern matching do par chave/valor.

FUNÇÃO 3

Achatar a árvore (flatten)

Converte aninhamento em chaves compostas com ponto. Listas usam índice.

Antes

```
%"usuario" ⇒ %{
  "nome" ⇒ "Ana"
}
```

Depois

```
%"usuario.nome" ⇒ "Ana"
"pedidos.0.preco" ⇒ 45.90
```

```
defp achatar(mapa, prefixo) when is_map(mapa) do
  # para cada chave, junta prefixo + chave e desce
end
defp achatar(lista, prefixo) when is_list(lista) do
  # usa índice 0, 1, 2… no caminho
end
defp achatar(valor, prefixo), do: [{prefixo, valor}]
```

MECANISMO

Como a recursão percorre a árvore

1. Recebe um nó (mapa, lista ou folha)
2. O **pattern matching** escolhe a cláusula certa
3. Se for composto, aplica a função em **cada filho**
4. Se for folha, aplica o **caso base** e para
5. Os resultados sobem e montam a nova estrutura

> *A forma dos dados define o fluxo — o mesmo espírito de "declarar regras" do paradigma funcional.*

ENTREGA PRÁTICA

Como rodar no glot.io

1. Acesse **https://glot.io/new/elixir**
2. Cole o conteúdo de `transformador.exs`
3. Clique em **Run**
4. Observe no console: original, rename, extract, flatten e extras

Saídas esperadas (resumo)

- Chaves em inglês após `renomear_chaves`
- `[45.9, 3.5]` ao extrair `"preco"`
- Mapa plano com caminhos `usuario.contato.email`, etc.

VISÃO DE PARADIGMAS

Funcional × outras abordagens

Imperativo / OO típico

- Loops e variáveis mutáveis
- `instanceof` / `typeof` / ifs
- Objetos com estado

Funcional (este trabalho)

- Recursão + casos base
- Cláusulas + guards
- Dados imutáveis transformados

O tema F3 força exatamente essa diferença: **descrever a estrutura** e deixar o matching decidir o caminho.

FECHAMENTO

Conclusões

- Estruturas tipo JSON são árvores — recursão é a ferramenta natural
- Elixir expressa mapa / lista / folha com cláusulas distintas
- Renomear, extrair e achatar são transformações puras compostas
- Pattern matching substitui a checagem explícita de tipos
- glot.io permite validar o código sem instalar toolchain local

OBRIGADO

Perguntas?

F3 — Transformador de estrutura tipo JSON

Arquivos da entrega

transformador.exs — código para glot.io

apresentacao.html — slides interativos

UNICID · Paradigmas de Linguagens de Programação

Prof. Cid Rodrigues De Andrade